

STA — Static Timing Analysis

🔥 시뮬레이션 없이 모든 timing path의 지연을 정적으로 계산해 setup/hold 등 타이밍 제약 만족 여부를 검증하는 sign-off 기법 · 🗂️ P1, 지원서연계 · 🧠 메모리 브릿지: DDR/HBM 같은 고속 인터페이스는 DQ-DQS setup/hold 마진, write/read leveling, source-synchronous 타이밍이 곧 성능. Peri/Base Die 디지털 로직의 timing closure도 PrimeTime sign-off.

핵심 개념

- **setup**: 데이터가 clock edge **이전** t_{su} 만큼 안정. **hold**: clock edge **이후** t_h 만큼 안정.
- **Setup 식**: $T_{clk} + skew \geq t_{cq} + t_{comb(max)} + t_{su}$. → **Setup slack** = $T_{clk} + skew - (t_{cq} + t_{comb_max} + t_{su})$. 주파수 한계, clock 늦추면 해결.
- **Hold 식**: $t_{cq} + t_{comb(min)} \geq t_h + skew$. → **Hold slack** = $(t_{cq} + t_{comb_min}) - (t_h + skew)$. 주기와 무관, clock 늦춰도 안 풀림 → buffer/delay 추가로 fix. 위반 시 chip dead.
- **slack = required - arrival**. +면 met, -면 violation.
- **clock skew**: launch FF와 capture FF의 clock 도착 시각 공간적 차이. useful skew는 setup 도움/hold 악화.
- **jitter**: clock edge의 시간적 변동(cycle-to-cycle). **uncertainty = skew+jitter+margin** → setup은 빼고 hold는 더함(pessimism).
- **false path**: 논리적으로 불가능/타이밍 안 잡는 경로(비동기 crossing, config). set_false_path.
- **multicycle path**: 데이터가 여러 cycle 써도 되는 경로. set_multicycle_path N.
- **OCV/derating**: launch/capture path에 P/V/T 변동 derate 적용. AOCV/POCV(통계적)로 과도 pessimism 완화.
- **recovery/removal**: 비동기 reset의 타이밍 체크. recovery=reset 해제가 clock edge **전에** 안정($\approx$ setup), removal=reset이 edge **후까지** 유지($\approx$ hold). 위반 시 메타.
- **timing closure/ECO**: 위반 반복 수정. ECO=post-route 국소 수정(resize/buffer/reroute). sign-off STA=PrimeTime(+SI crosstalk, POCV).

예상 면접 Q&A

Q1. setup time과 hold time을 정의하고 각각의 식을 쓰세요.

A. setup은 데이터가 clock edge 이전 t_{su} 동안, hold는 clock edge 이후 t_h 동안 안정해야 한다는 제약입니다. Setup은 $T_{clk} + skew \geq t_{cq} + t_{comb_max} + t_{su}$ (slack = $T_{clk} + skew - (t_{cq} + t_{comb_max} + t_{su})$). Hold는 $t_{cq} + t_{comb_min} \geq t_h + skew$ (slack = $(t_{cq} + t_{comb_min}) - (t_h + skew)$). Setup은 가장 느린 path, Hold는 가장 빠른 path가 결정합니다. - L, 꼬리질문: 둘 중 더 위험한 위반은? → Hold. 주기와 무관해 clock을 늦춰도 안 풀리고, silicon에서 발견되면 chip이 죽음. Setup은 주파수만 낮추면 임시 회피 가능. - 🧠 메모리 브릿지: DDR write 시 DQS로 DQ를 캡처할 때 DQ가 DQS 대비 setup/hold 윈도우(data eye) 안에 들어와야 → leveling/training으로 맞춤.

Q2. slack이 무엇이고 음수면 어떻게 해석하나요?

A. slack = required time – arrival time. 신호가 도착해야 하는 한계(required) 대비 실제 도착(arrival)의 여유입니다. 양수면 마진이 남아 타이밍 만족, 음수면 위반(WNS=worst negative slack, TNS=총합). Setup slack 음수는 path가 너무 느린 것, Hold slack 음수는 path가 너무 빠른 것입니다. - L, 꼬리질문: WNS만 보면 되나? → 아님. TNS/위반 path 수도 봐야 closure 난이도와 ECO 규모를 안다. - 🧠 메모리 브릿지: Peri 로직 다수 path를 한꺼번에 정적 분석 → 시뮬 없이 전수 검증이 STA의 강점.

Q3. clock skew와 jitter, uncertainty의 차이는?

A. skew는 같은 clock이 두 FF에 도착하는 공간적(정적) 시각 차이, jitter는 같은 지점에서 edge가 시간적으로(동적) 흔들리는 변동입니다. STA에선 둘과 margin을 묶어 clock uncertainty로 모델링하고, setup 체크엔 빼서(available time 감소), hold 체크엔 더해서(pessimism) 보수적으로 봅니다. useful skew(capture clock을 일부러 늦춤)는 setup을 돕지만 hold를 악화시킵니다. - L, 꼬리질문: jitter는 누가 만드나? → PLL/DLL, 전원 노이즈, crosstalk. 그래서 PI/SI가 timing과 연결. - 🧠 메모리 브릿지: DDR 고속에선 clock/strobe jitter가 data eye를 좁혀 마진을 직접 갉아먹음.

Q4. false path와 multicycle path를 설명하세요.

A. false path는 실제로 데이터가 전파되지 않거나 타이밍을 잡을 필요가 없는 경로(비동기 도메인 crossing, mutually exclusive MUX 경로, 정적 config)로 set_false_path로 STA에서 제외합니다. multicycle path는 데이터가 한 cycle이 아니라 N cycle에 한 번만 캡처되면 되는 경로로 set_multicycle_path N을 줘서 setup을 N배 주기로 완화합니다(이때 hold MCP도 함께 설정해야 hold가 깨지지 않음). - L, 꼬리질문: false path를 잘못 걸면? → 진짜 위반을 숨김. 그래서 CDC/구조 근거로만 걸고 리뷰. - 🧠 메모리 브릿지: 코어↔PHY 비동기 crossing은 false path로 빼고 CDC sign-off로 따로 검증 — STA와 CDC가 짝.

Q5. OCV와 derating, AOCV/POCV는 무엇인가요?

A. 같은 chip 안에서도 위치별 P/V/T가 달라 동일 셀의 지연이 다릅니다(On-Chip Variation). STA는 launch path는 느리게/capture path는 빠르게(또는 반대로) derate factor를 곱해 worst를 봅니다. 단순 OCV는 과도하게 pessimistic이라, path 깊이에 따라 변동이 평균화되는 걸 반영한 AOCV, 셀별 통계 분포를 쓰는 POCV(statistical)로 pessimism을 줄여 더 현실적 마진을 얻습니다. - L, 꼬리질문: corner는 왜 여러 개 보나? → SS/FF/TT × 전압 × 온도 조합(MCMM)에서 setup은 보통 slow corner, hold는 fast corner가 worst. - 🧠 메모리 브릿지: 메모리 동작 전압/온도 범위가 넓어 corner 수가 많고 hold(fast/저온) 마진 관리가 중요.

Q6. recovery/removal 체크가 무엇이고 setup/hold와 어떻게 다른가요?

A. 비동기 reset(또는 set)의 해제(deassert)가 clock과 충돌하지 않게 하는 체크입니다. recovery는 reset 해제가 clock edge 이전 일정 시간 안정해야 한다는 것으로 **setup과 유사**, removal은 reset이 clock edge 이후 일정 시간 유지돼야 한다는 것으로 **hold와 유사**합니다. 차이는 대상이 동기 데이터가 아니라 비동기 control(reset)이고, 위반하면 FF가 reset 해제 순간 metastable이 될 수 있다는 점입니다 → reset synchronizer로 "async assert, sync deassert". - L, 꼬리질문: 그래서 reset 해제를 동기화하는 이유? → 모든 FF가 같은 clock edge에 동시에 reset 해제되도록 해 recovery/removal과 메타를 막음(RDC와 연결). - 🧠 메모리 브릿지: 메모리 IP의 다중 reset 도메인 해제 타이밍도 recovery/removal·RDC로 sign-off.

Q7. timing closure와 ECO, PrimeTime sign-off를 설명하세요. (지원서연계)

A. timing closure는 합성~배치배선 후 남은 setup/hold/transition/DRV 위반을 반복적으로 없애 모든 corner·mode에서 $\text{slack} \geq 0$ 을 만드는 과정입니다. ECO는 placement·routing을 크게 안 흔들고 셀 resize, buffer 삽입, VT swap, reroute로 국소 수정하는 것이고, 최종 sign-off는 Synopsys PrimeTime(SI crosstalk delay, POCV, noise 포함)로 합니다. 저는 sign-off 방법론·EDA 자동화를 다루며 timing/CDC/lint 결과를 통합·자동 분류해 closure 효율을 높이는 일을 했습니다. - L, 꼬리질문: setup ECO와 hold ECO 차이? → setup은 셀 upsize/VT down/로직 단순화(빠르게), hold는 buffer/delay cell 삽입(느리게). hold는 거의 모든 corner에서 동시에 잡아야. - 🧠 메모리 브릿지: Peri/Base Die 로직의 PrimeTime sign-off와 인터페이스 타이밍 budget이 메모리 동작 주파수를 결정.

30초 암기 요약

- $\text{setup} = \text{edge 전 안정}$, $\text{hold} = \text{edge 후 안정}$. $\text{Setup slack} = T_{\text{clk}} + \text{skew} - (t_{\text{cq}} + t_{\text{comb_max}} + t_{\text{su}})$.
 - $\text{Hold slack} = (t_{\text{cq}} + t_{\text{comb_min}}) - (t_{\text{h}} + \text{skew})$. **주기 무관**, clock 늦춰도 안 풀림 → buffer로 fix. 위반=chip dead.
 - $\text{slack} = \text{required} - \text{arrival}$. WNS/TNS로 closure 난이도 판단.
 - $\text{skew} = \text{공간적 정적}$, $\text{jitter} = \text{시간적 동적}$. $\text{uncertainty} = \text{skew} + \text{jitter} + \text{margin}$ (setup-, hold+).
 - false path=타이밍 제외, MCP=N cycle 완화(hold MCP도 같이).
 - OCV/derating으로 P/V/T 변동 반영, AOCV/POCV로 pessimism 완화. setup=slow corner, hold=fast corner.
 - $\text{recovery} \approx \text{setup}$, $\text{removal} \approx \text{hold}$ (비동기 reset 해제). reset synchronizer로 async assert/sync deassert.
 - sign-off=PrimeTime(+SI/POCV). ECO: setup=upscale, hold=buffer 삽입.
-